Developing Requirements

Adapted from:

Timothy Lethbridge and Robert Laganiere,

Object-Oriented Software Engineering –

Practical Software Development using UML and Java, 2005

(chapter 4)

4.1 Domain Analysis

***Domain analysis* is the process by which a software engineer learns about the application domain to better understand the problem.**

- The *domain* is the general field of business or technology in which the clients will use the software
- A *domain expert* is a person who has a deep knowledge of the domain

Benefits of performing domain analysis:

- Faster development
- Better system
- Anticipation of extensions

Suggested Structure of a Domain Analysis document

A. Introduction

B. Glossary

C. General knowledge about the domain

D. Customers and users

E. The environment

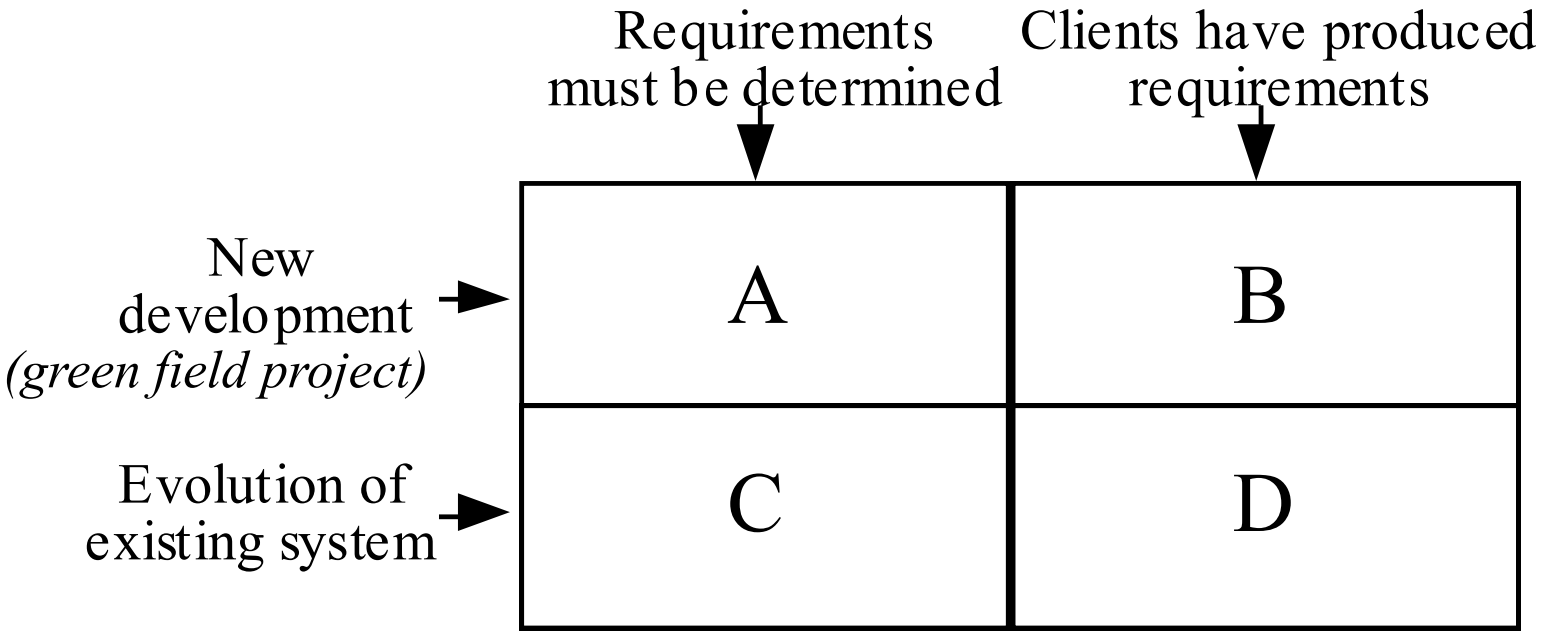
F. Tasks and procedures currently performed

G. Competing software

H. Similarities to other domains

4.2 The Starting Point for Software Projects

Four broad categories of software projects:



4.3 Defining the Problem and the Scope

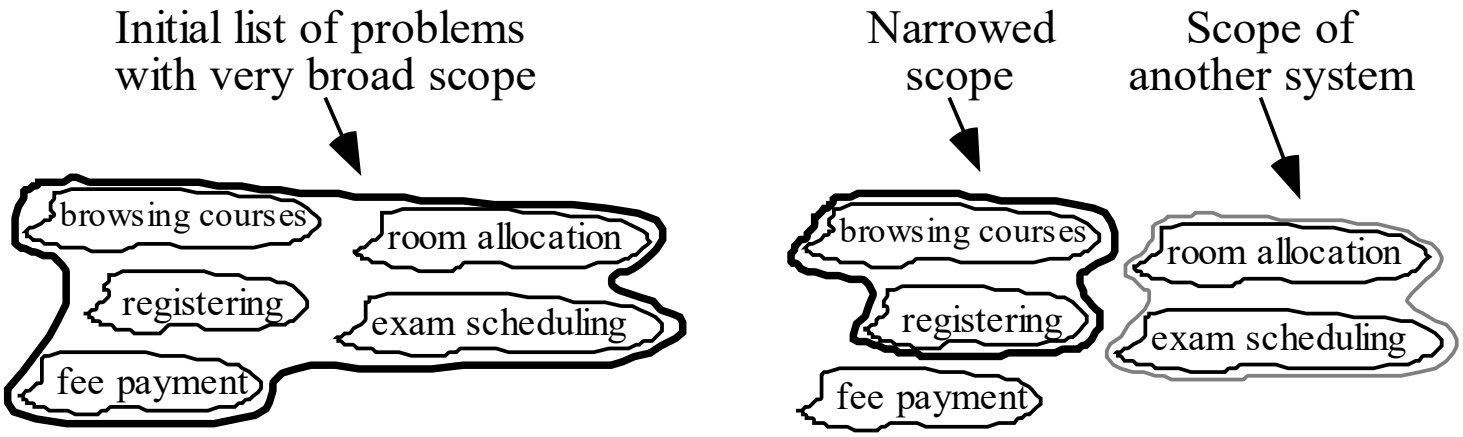
- **In order to determine the requirements, the first step is to work out an initial definition of the *problem* to be solved.**
- **A problem can be expressed as:**
  - A *difficulty* the users or customers are facing,
  - Or as an *opportunity* that will result in some benefit such as improved productivity or sales.
- **The solution to the problem normally will entail developing software.**
- **A good *problem statement* is short and succinct - one or two sentences is best.**
  - Example - a new student registration system:
    - —"The system will allow students to register for courses, and change their registration, as simply ad rapidly as possible. It will help students achieve their personal goals of obtaining their degree in the shortest reasonable time while taking courses that they find most interesting and fulfilling."

Defining the Scope

An important objective is to narrow the *scope* by defining a more precise problem

- List all the things you might imagine the system doing
  - —Exclude some of these things if too broad
  - —Determine high-level goals (of the user / customer) if too narrow

Example: A university registration system



4.4 What is a Requirement

A *requirement* is a statement describing either 1) an aspect of what the proposed system must do, or 2) a constraint on the system's development. In either case, it must contribute in some way towards adequately solving the customer's problem; the set of requirements as a whole represents a negotiated agreement among all stakeholders.

- A requirement is a short and concise piece of information

A collection of requirements is a *requirements document*.

4.5 Types of Requirements

Functional requirements

- Describe *what* the system should do

Non-functional requirements

- C*onstraints* that must be adhered to during development

Functional requirements

Functional requirements describe the *services* provided for the *users* and for *other systems*.

Functional requirements can describe:

- What *inputs* the system should accept
- What *outputs* the system should produce
- What data the system should *store* that other systems might use
- What *computations* the system should perform
- The *timing and synchronization* of the above

Non-functional requirements

All must be verifiable

Three main types

1. Categories reflecting *quality attributes*:
   - —Response time
   - —Resource usage
   - —Reliability
   - —Availability
   - —Recovery from failure
   - —Maintainability
   - —Reusability

Non-functional requirements

2. Categories constraining the *environment and technology* of the system:
   —Platform
   —Technology to be used

3. Categories constraining the *project plan and development methods*:
   —Development process (methodology) to be used
   —Cost and delivery date
      - Often put in contract or project plan instead

4.6 Use-Cases describing how the user will use the system

- **A *use case* is a description of a set of sequences of actions, including variants, that a system performs that yields an observable result of value to a particular actor [BRJ99].**
  - The objective of *use case analysis* is to model the system
    … from the point of view of how users interact with this system
    … when trying to achieve their objectives.
  - A *use case model* consists of
    — a set of use cases
    — a set of actors (an *actor* is a coherent set of roles that users of use cases play when interacting with the use cases [BRJ99])
    — an optional description or diagram indicating how they are related
  - To make a use case model understandable, you should group similar variant sequences of actions into one use case.

Use cases

- In general, a use case should cover the *full sequence of steps* from the beginning of a task until the end.
- A use case should describe the *user's interaction* with the system ...
  —not the computations the system performs.
- A use case should be written so as to be as *independent* as possible from any particular user interface design.
- A use case should only include actions in which the actor interacts with the computer.

Scenarios

- **A scenario is an *instance* of a use case, involving**
  - —a specific actor instance,
  - —at a specific time and
  - —using specific data.

How to describe a single use case

A. Name: Give a short, descriptive name to the use case.

B. Actors: List the actors who can perform this use case.

C. Goals: Explain what the actor or actors are trying to achieve.

D. Preconditions: State of the system before the use case.

E. Description: Give a short informal description.

F. Related use cases

G. Steps: Describe each step (using a 2-column format: actor actions, system responses)

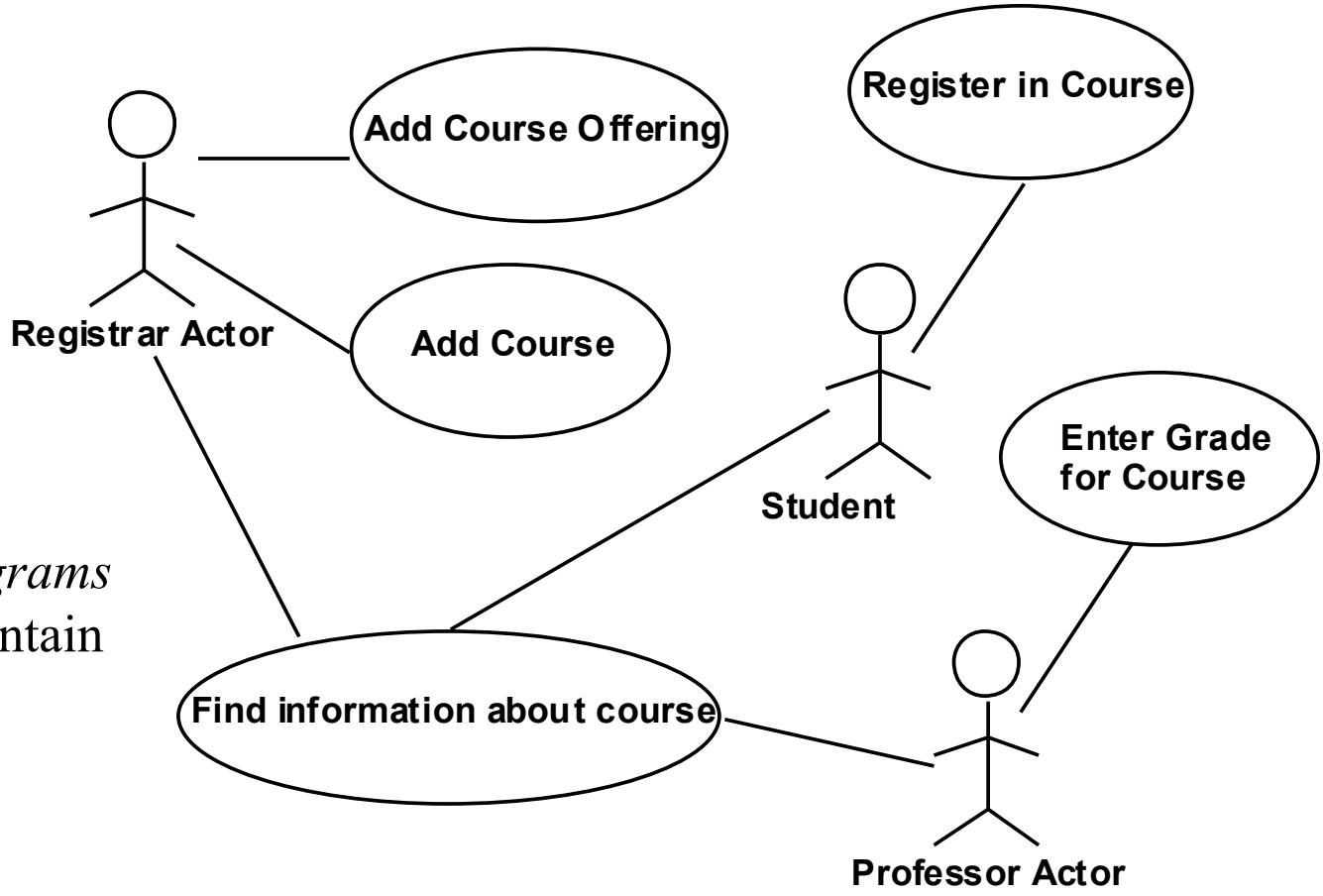
H. Post-conditions: What state is the system in following the completion of the use case.

- Only the name (A) and the steps (G) are essential
  - —In a simplified description you may omit the other components.

A simple use case diagram

Graphically
• a *use case* is rendered as an ellipse,
• an *actor* is rendered as a stick person.

Use case diagrams commonly contain [BRJ99]:
• actors,
• use cases,
• associations, generalizations and dependencies.



Extensions

- Used to make *optional* interactions explicit or to handle *exceptional* cases.
  - —By creating separate use case extensions, the description of the basic use case remains simple.
  - —In the use case extension you must
    - either list all the steps from the beginning of the use case to the end, including the handling of the unusual situation,
    - or indicate which step is the extension point (the point at which the extension changes the basic sequence).

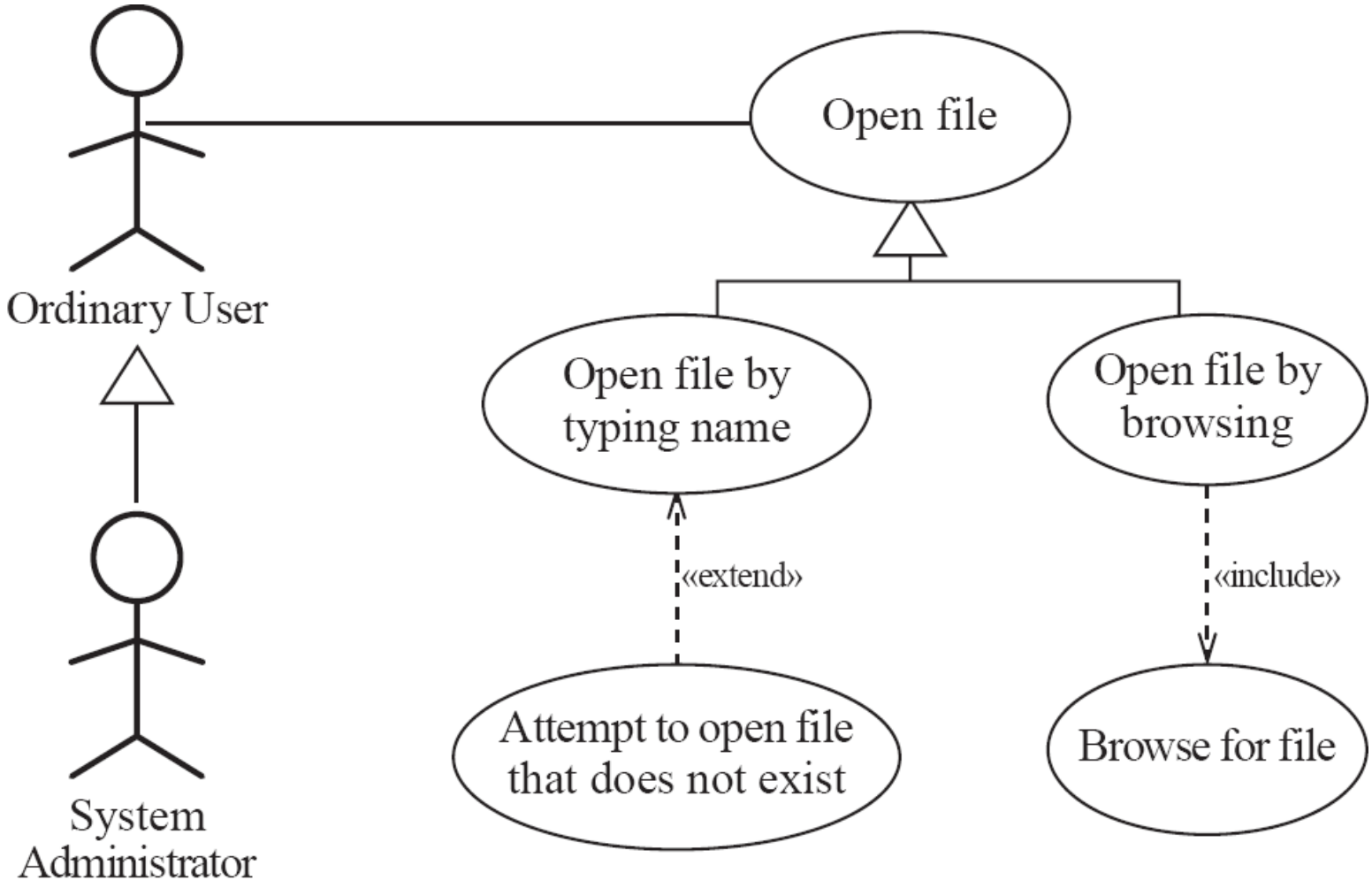
Generalizations

- *Generalizations* (/ *specializations*) work the same way as in a class diagram, and use the same triangle symbol.
  - —A generalized use case represents *several similar* use cases.
  - —One or more specializations provides details of the similar use cases.

Inclusions

- Allow one to express *commonality* between several different use cases.
- Are included in other use cases
  - —Even very different use cases can share sequence of actions.
  - —Enable you to avoid repeating details in multiple use cases.
- Represent the performing of a *lower-level task* with a lower-level goal.

Example of generalization, extension and inclusion



Example description of a use case diagram

Use case: **Open file**

Related use cases:
Generalization of:
- Open file by typing name
- Open file by browsing

Steps:

Actor actions	**System responses**
1. Choose ‘Open…’ command	2. File open dialog appears
3. Specify filename	
4. Confirm selection	5. Dialog disappears

Example (continued)

Use case: **Open file by typing name**

Related use cases:
Specialization of: Open file

Steps:

Actor actions	**System responses**
1. Choose ‘Open…’ command	2. File open dialog appears
3a. Select text field	
3b. Type file name	
4. Click ‘Open’	5. Dialog disappears

Example (continued)

Use case: **Open file by browsing**

Related use cases:
Specialization of: Open file
Includes: Browse for file

Steps:

Actor actions	**System responses**
1. Choose ‘Open…’ command	2. File open dialog appears
3. Browse for file	
4. Confirm selection	5. Dialog disappears

Example (continued)

Use case: **Attempt to open file that does not exist**

Related use cases:
Extension of: Open file by typing name

Actor actions	**System responses**
1. Choose ‘Open…’ command	2. File open dialog appears
3a. Select text field 3b. Type file name	
4. Click ‘Open’	5. System indicates that file does not exist
6. Correct the file name	
7. Click ‘Open’	8 Dialog disappears

Example (continued)

Use case: **Browse for file (inclusion)**

Steps:

Actor actions	**System responses**
1. If the desired file is not displayed, select a directory	2. Contents of directory is displayed
3. Repeat step 1 until the desired file is displayed	
4. Select a file	

The modeling processes: Choosing use cases on which to focus

- Often one use case (or a very small number) can be identified as *central* to the system
  - For example, in an airline reservation system the central use case will be 'Reserve a seat on a flight'
    - » The entire system can be built around this particular use case
- There are also other reasons for focusing on particular use cases:
  - —Some use cases will represent a high *risk* because for some reason their implementation is problematic
  - —Some use cases will have high *political or commercial value*

The benefits of basing software development on use cases

- Use cases can help to define the *scope* of the system.
- Use cases are often used to *plan* the development process.
- Use cases are used to both develop and validate the requirements.
- Use cases can form the basis for the definition of test cases.
  - —A *test case* is a specification of one case to test the system, including what to test, with which input, expected result, and under which conditions [JBR99].
    - A test case specifies a testing scenario.
- Use cases can be used to structure user manuals.

The benefits of basing software development on use cases

- The basic *integration testing strategies* (top-down and bottom-up) are applicable where the design is a hierarchy of modules or subsystems.
  - —In an OO system the basic modules to be tested are object classes that are instantiated as objects.
  - —However, in general, in an OO system there is no obvious 'top' to the system [Som01].
- **Use case or scenario based testing is often the most effective integration testing strategy for OO systems [JBR99,Som01,Som16].**
- In this approach the software engineer identifies testing scenarios from use cases.
  - —A set of test cases can be identified for each use case.

4.7 Some Techniques for Gathering Requirements

Observation

- Read documents and discuss requirements with users
- Shadowing important potential users as they do their work
  —ask the user to explain everything he or she is doing
- Session videotaping

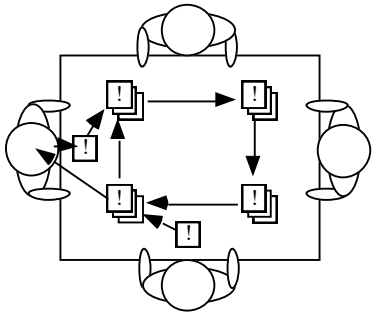
Interviewing

- Conduct a series of interviews
  —Ask about specific details
  —Ask about the stakeholder's vision for the future
  —Ask if they have alternative ideas
  —Ask for other sources of information
  —Ask them to draw diagrams

Gathering Requirements...

Brainstorming

- Appoint an experienced moderator
- Arrange the attendees around a table
- Decide on a 'trigger question'. Examples of trigger questions:
  - What features are important in the system?
  - What future sources of data should we anticipate?
  - What outputs should be produced by the system?
  - What classes do we need in our domain model?
- Ask each participant to write an answer and pass the paper to its neighbour



Joint Application Development (JAD) is a technique based on intensive brainstorming sessions

Gathering Requirements...

Prototyping

- The simplest kind: *paper prototype*.
  - —a set of pictures of the system that are shown to users in sequence to explain what would happen
- The most common: a mock-up of the system's UI
  - —Written in a rapid prototyping language
  - —Does *not* normally perform any computations, access any databases or interact with any other systems
  - —May prototype a particular aspect of the system

4.8 Types of Requirements Document

Two extremes which should be avoided:

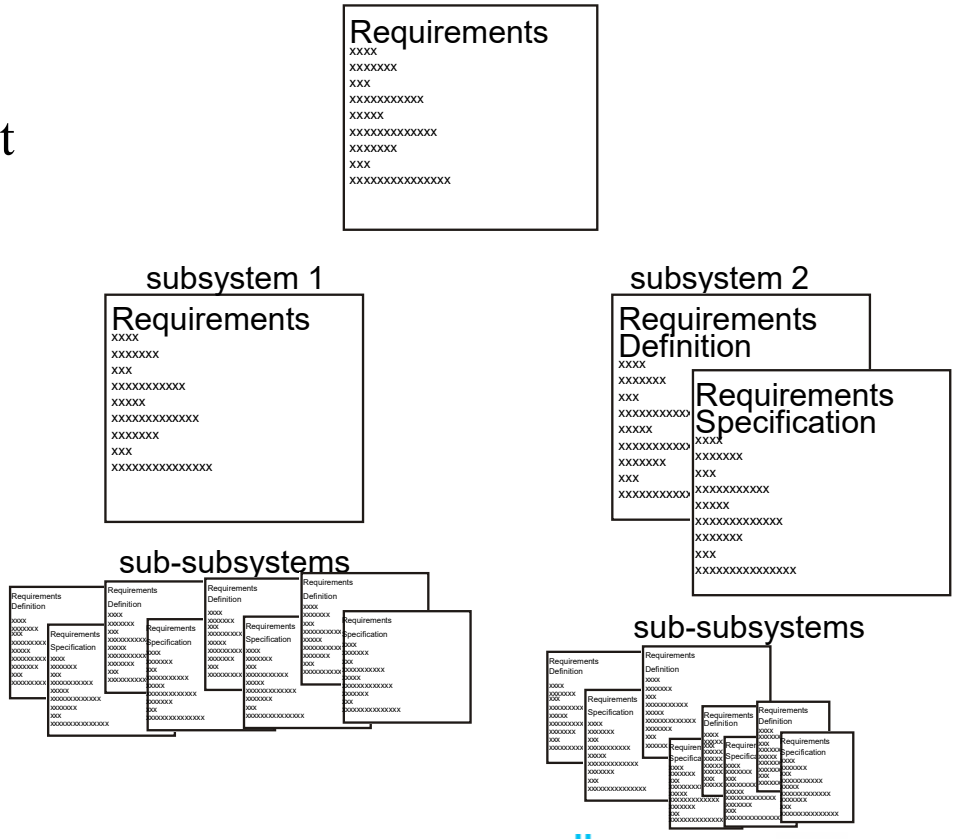
•An informal outline of the requirements using a few paragraphs or simple diagrams

• A long list of specifications that contain thousands of pages of intricate detail

• The term *requirements definition* normally refers to a less detailed, higher-level document.

• A *requirements specification* is a more detailed and precise document.

- Requirements documents for large systems are normally arranged in a hierarchy



Level of detail required in a requirements document

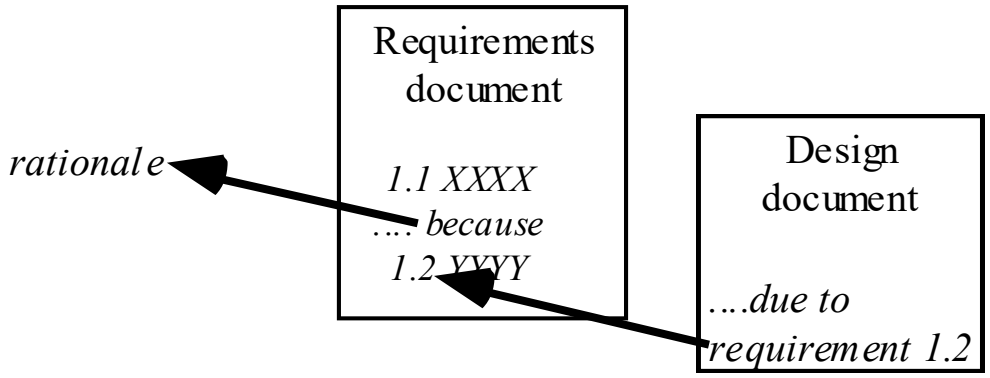
- How much detail should be provided depends on:
  - —The size of the system
  - —The need to interface to other systems
  - —The readership
  - —The stage in requirements gathering
  - —The level of experience with the domain and the technology
  - —The cost that would be incurred if the requirements were faulty

4.9 Reviewing Requirements

- **Each individual requirement should be carefully reviewed. In order to be acceptable each individual requirement should:**
  - —Have **benefits that outweigh the costs** of development
  - —Be **important** for the solution of the current problem
  - —Be expressed using a **clear and consistent notation**
  - —Be **unambiguous**
  - —Be **logically consistent**
  - —Lead to a system of **sufficient quality**
  - —Be **realistic** with available resources
  - —Be **verifiable**
  - —Be uniquely **identifiable**
  - —**Does not over-constrain the design** of the system

Requirements documents...

- The requirements document should be:
  - —sufficiently complete
  - —well organized
  - —clear
  - —agreed to by all the stakeholders
- In a design document it should be possible to say which requirement is being implemented by a given aspect of the design.
  - —This quality is called *traceability*.



Suggested structure of a requirements document

A. Problem

- A succinct description of the problem to be solved

B. Background information

- References to domain analysis documents, standards, related systems

C. Environment and system models

- The context in which the system runs, a global overview of the subsystems, the hardware support, etc.

D. Functional Requirements

E. Non-functional requirements

Additional references

[BRJ99] G. Booch, J. Rumbaugh and I. Jacobson. *The Unified Modeling Language User Guide*. Addison-Wesley, 1999.

[JBR99] I. Jacobson, G. Booch and J. Rumbaugh. *The Unified Software Development Process*. Addison-Wesley, 1999.

[Som01] I. Sommerville. *Software Engineering* (6th edition). Addison-Wesley, 2001.

[Som16] I. Sommerville. *Software Engineering* (10th edition). Addison-Wesley, 2016.